

CHI Program Analytics - Project Case Study

Notes on what I built, how, and why. Written up mainly so I have the details straight for interviews and for explaining it to CHI leadership.

1. Where this started

I work as a Data Systems & Operations Analyst at Chicago Help Initiative (CHI), supporting the Job Room / Workplace Support Center. Client tracking for that program lived in a manual, multi-tab Google Sheet I built and maintained myself: client intake, visit logs, document readiness, services provided (resumes, phones, bus passes), and a “Number of Visitors Analysis” tab used for monthly reporting.

That sheet can answer “how many clients did we serve this month.” It can’t answer “what does it cost per client, and is that trending well.” That’s the gap this project fills.

I took the real (anonymized) data behind that manual system, rebuilt it as a tested data warehouse, and added CHI’s actual financial data on top to get real cost-per-client numbers instead of a visit count.

2. Real data vs. synthetic

The raw export was small, about 99 rows total across clients and visits. I could’ve used fake data instead, but decided to use:

1. The real, anonymized CHI client and visit data
2. CHI’s real public IRS Form 990 filings (2012-2025, from ProPublica Nonprofit Explorer, EIN 45-2542979) for the financial side

Kept it real and anonymized rather than fully synthetic so every number in this project traces back to something actual, not made up.

3. Tools

Tool	What it did
Excel / Google Sheets	Original source system, the raw intake/visit export
Python	<code>anonymize_chi_data.py</code> strips PII and reshapes the raw export into clean seed CSVs
Snowflake	Cloud warehouse (free trial), database <code>CHI_ANALYTICS</code> , schema <code>ANALYTICS</code>
dbt Core	Turns raw seeds into tested, documented SQL models, generates the lineage graph
SQL	Joins, monthly rollups, cost allocation math, YoY trend logic
ProPublica Nonprofit Explorer	Public source for CHI’s 990 filings

Went with Snowflake + dbt specifically because it’s the pairing most analytics teams are actually running right now.

4. Privacy

The original export had real client names, phone numbers, emails, home addresses, and sensitive fields (SNAP status, SS card/birth certificate/state ID possession, free-text visit notes). None of that made it into the project. Before any modeling started, `anonymize_chi_data.py`:

- Removed entirely: names, emails, phones, addresses, free-text notes, “other specify” fields
- Collapsed SS card/birth cert/state ID into one `document_readiness_score` (0-3) so no individual document status is queryable per person

- Dropped SNAP status entirely
- Shuffled and reissued client IDs so they can't be traced back to the original sheet
- Pseudonymized volunteer names (Volunteer_A, Volunteer_B, ...)

Output: 75 anonymized client records, 50 anonymized visit records, saved as `seeds/dim_clients.csv` and `seeds/fact_visits.csv`.

5. Architecture

```
seeds/                                raw (already anonymized) source tables
  dim_clients.csv
  fact_visits.csv
  org_financials.csv                  real CHI 990 data, 2012-2025

models/staging/                       1:1 cleaned, typed, renamed views
  stg_clients.sql
  stg_visits.sql
  stg_org_financials.sql

models/intermediate/                 reusable aggregation logic
  int_monthly_operations.sql

models/marts/                        BI-ready, tested tables
  fct_program_operations.sql          -- monthly visits/clients/services
  fct_org_financial_trends.sql        -- YoY revenue/expense trend
  fct_cost_per_client_estimate.sql    -- the FP&A headline metric
```

Standard dbt layering (staging → intermediate → marts) instead of one big query, mainly because each layer has one job so bugs are easy to isolate, and every layer gets tested independently.

21 dbt tests run across the project: uniqueness and not-null checks on every primary key, referential integrity between facts and dimensions, and accepted-value range checks (like `document_readiness_score` needing to be 0-3). All 21 pass.

6. What I actually did, roughly in order

1. Exported the real Job Room intake/visit sheet to Excel
2. Wrote and debugged `anonymize_chi_data.py` (fixed a header-whitespace bug and a blank-header date column bug along the way) until it produced clean, PII-free seed CSVs
3. Pulled CHI's 990 data from ProPublica, built `seeds/org_financials.csv`
4. Set up a Snowflake trial account, database and schema, installed dbt Core + the Snowflake adapter, connected the two through `profiles.yml`
5. Built staging models to type and rename raw columns into clean views
6. Built `int_monthly_operations.sql` to roll client and visit data up to a monthly grain
7. Built the three marts, including the cost-estimate model driven by an adjustable variable (`job_room_expense_allocation_pct`, default 15%) instead of a hardcoded number
8. Wrote and ran 21 dbt tests, caught a real bug in the process (a boolean-vs-integer comparison in the monthly rollup that would've silently produced wrong counts)
9. Ran everything against the live Snowflake account: `dbt seed`, `dbt run`, `dbt test`
10. Fixed a data-recency issue where the current, still-in-progress month was showing a misleading partial drop and a null in `new_clients`. Fixed it at the model level (`where activity_month < date_trunc('month', current_date()) plus coalesce(new_clients, 0)`) so it stays correct going forward instead of needing a manual fix every month
11. Ran a sensitivity test on the cost-allocation assumption (section 8)
12. Built two ways to visualize the pipeline for comparison (section 9)

7. Where AI helped

I used Claude for a chunk of this: talking through the architecture before writing code, debugging real terminal errors as they came up, and as a second set of eyes on the SQL and Python. I wrote/ran everything myself and reviewed each piece before using it, but it's fair to say this project moved a lot faster with that help than it would have solo.

8. The FP&A metric and its limits

`fct_cost_per_client_estimate` computes cost per client served and cost per visit by allocating a share of CHI's total expenses (from the 990) to the Job Room program. Two limitations I chose to handle explicitly instead of ignoring:

1. 990 filings report organization-wide totals, not a per-program breakdown. CHI runs more than just Job Room, so I allocate a configurable share of total expenses to it via `job_room_expense_allocation_pct`, a documented assumption rather than a hardcoded guess.
2. 990 filings lag 12-18 months. I use the most recent year available as the cost proxy against current operational data instead of forcing an exact match that isn't possible.

Sensitivity test

To check the allocation assumption doesn't produce arbitrary results, I reran the model at two allocation rates:

Allocation %	Cost per client served	Cost per visit
15% (default)	\$1,958.21	\$979.10
20% (test)	\$2,610.94	\$1,305.47

$20\% \div 15\% = 1.333x$, and both cost metrics scaled by exactly that factor. That means the model responds linearly to its one adjustable assumption with no hidden bugs, which is what you'd want to confirm before trusting a number like this. Reverted back to the 15% default afterward.

9. Two ways to look at the same pipeline

Snowflake's native chart. Built a line chart on `fct_program_operations` with two series, total visits and unique clients served. The two lines track closely (roughly 1 visit per client) from January through May, then diverge starting in June, visits growing faster than unique clients. That means repeat engagement per client is going up, peaking around 1.8 visits per client in July. Different (and more useful) story than just "more people showed up."

dbt's lineage graph (`dbt docs generate && dbt docs serve`). Shows the pipeline itself: every model as a node, every `ref()` as an arrow, generated automatically from the actual SQL. Tracing `fct_program_operations` backward shows exactly which seed files and transformations produced that table.

	Snowflake chart	dbt lineage graph
Shows	the data	the pipeline
Audience	funder / supervisor	engineer / analyst
Answers	what happened	how it was built, and can you trust it

10. What this shows

For interviews: end-to-end pipeline ownership (messy source to tested, documented warehouse), privacy/governance judgment applied before any analysis, an FP&A cost model built on incomplete data with a documented and tested assumption instead of a black box number, and working knowledge of the Snowflake + dbt stack.

For CHI leadership: I took the manual system I built for the Job Room program and modernized it in my own time, then used CHI's own public financial filings to show what cost per client actually looks like, with a transparent way to test that number if the expense allocation assumption changes. Worth being clear that CHI's live systems don't run on this stack, this is a personal project built on anonymized exports.

11. Resume framing

“Rebuilt a manual nonprofit client-tracking system as a tested Snowflake + dbt warehouse (21 automated data tests, auto-generated lineage docs); added FP&A unit-economics analysis (cost per client served, cost per visit) using the organization’s public IRS filings, including a documented, sensitivity-tested cost-allocation methodology.”